

KAGGLE REGRESSION CHALLENGE

Predicting a Heavy-Tailed Target

A Sign-Coupled Two-Stage Model — How submission_ts_signc_s06.csv won

0.07434

FINAL private R^2 (winner)

sign-coupled

magnitude + borrowed sign

5-fold

CV: read the median

$\pm 3,300$

prediction range

Predict a continuous target from 15 anonymous features

Data

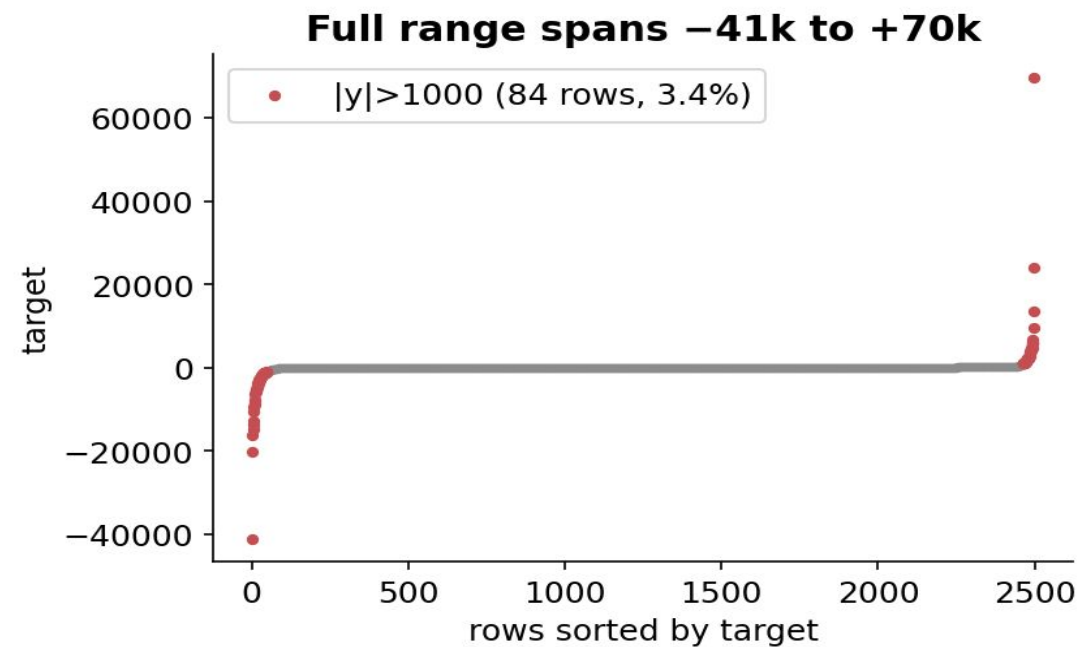
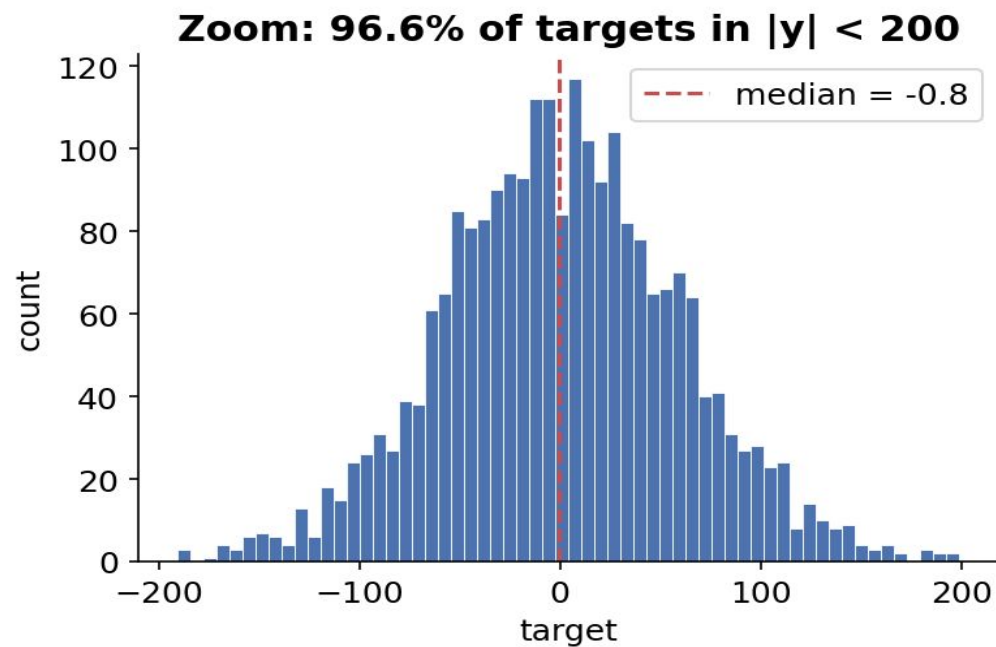
2,500 train rows · 2,500 test rows. 15 numeric features x_0 – x_{14} with no domain meaning.
~1–1.5% values missing per feature.

Metric

R^2 on the RAW target. 1.0 = perfect, 0 = predicting the mean, < 0 = worse than the mean. Squared errors → the tail dominates.

The one fact that controls everything

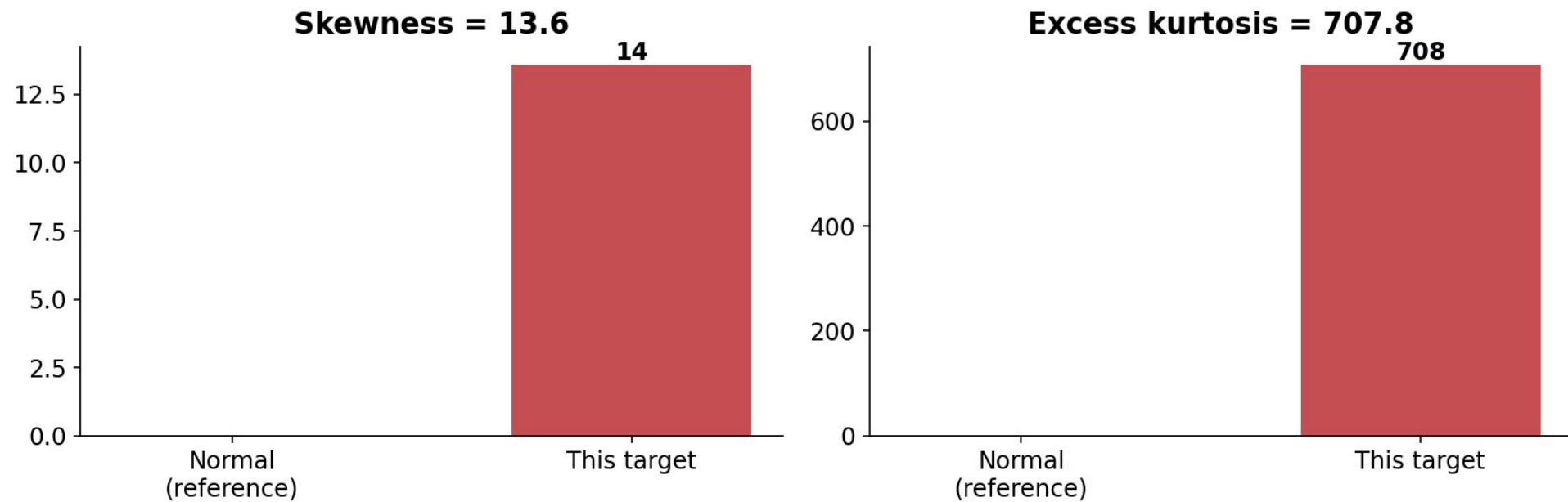
The most important fact: the target is extremely heavy-tailed



Why it matters: R^2 is built on squared errors, so the ~84 giant rows (3.4%) dominate the score. Model the tail acceptably and the score moves; nail the other 96% but botch the tail and it barely budes.

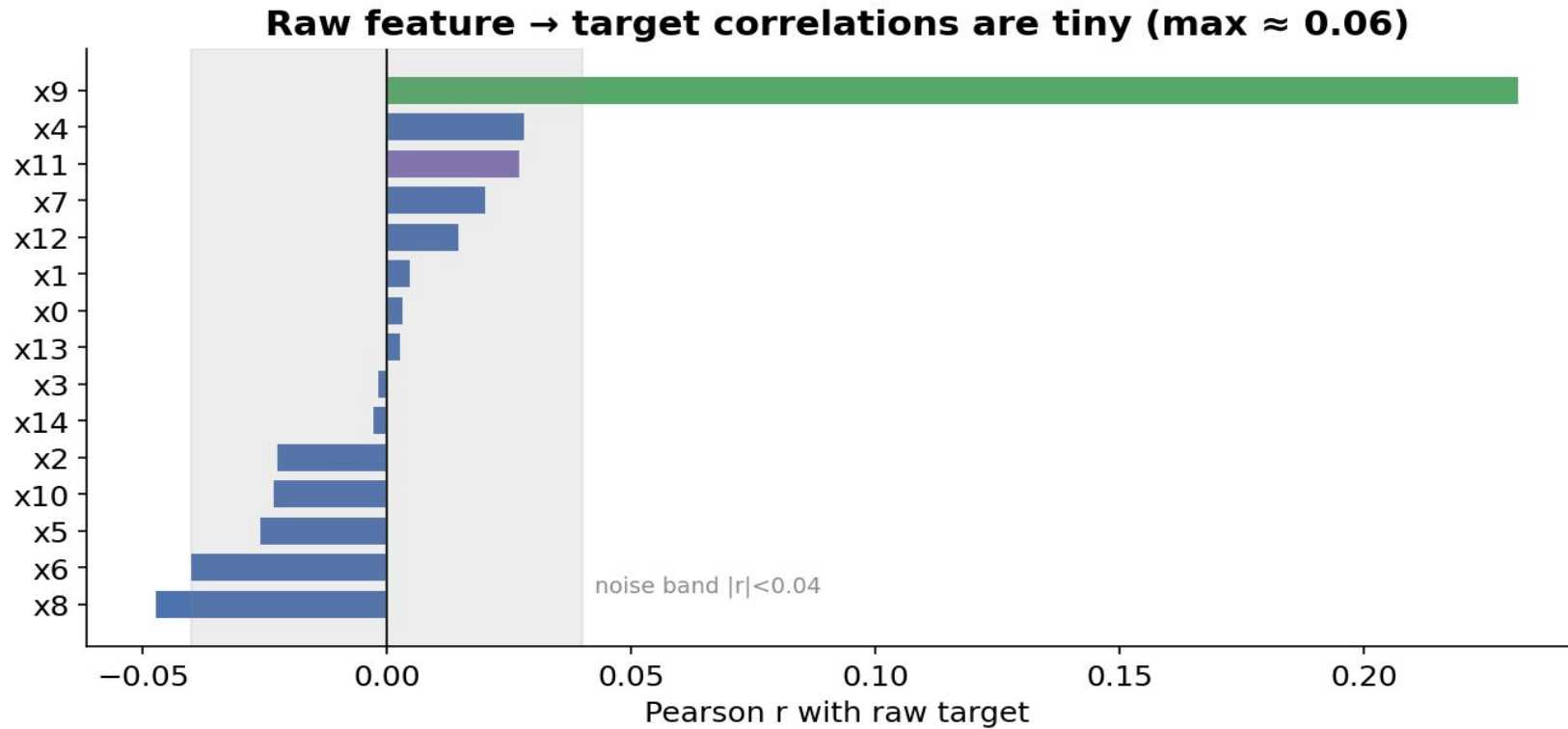
Off-the-chart skew and kurtosis

A bell curve scores 0 on both — squared-error R^2 will be ruled by the tail



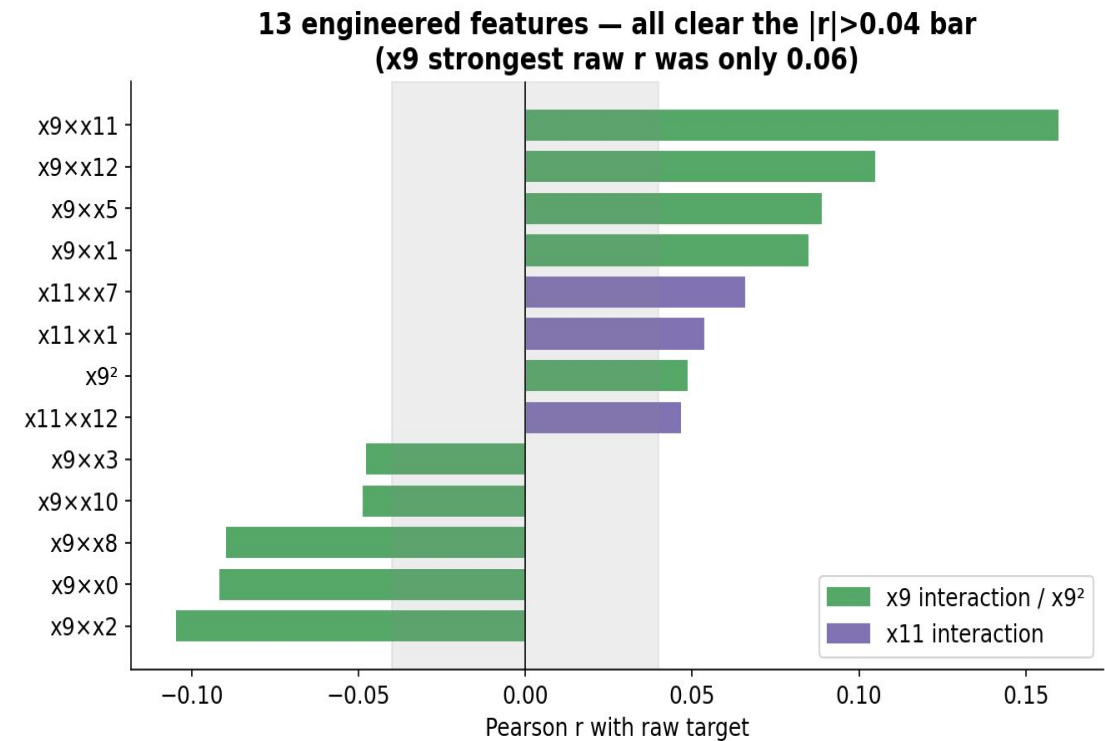
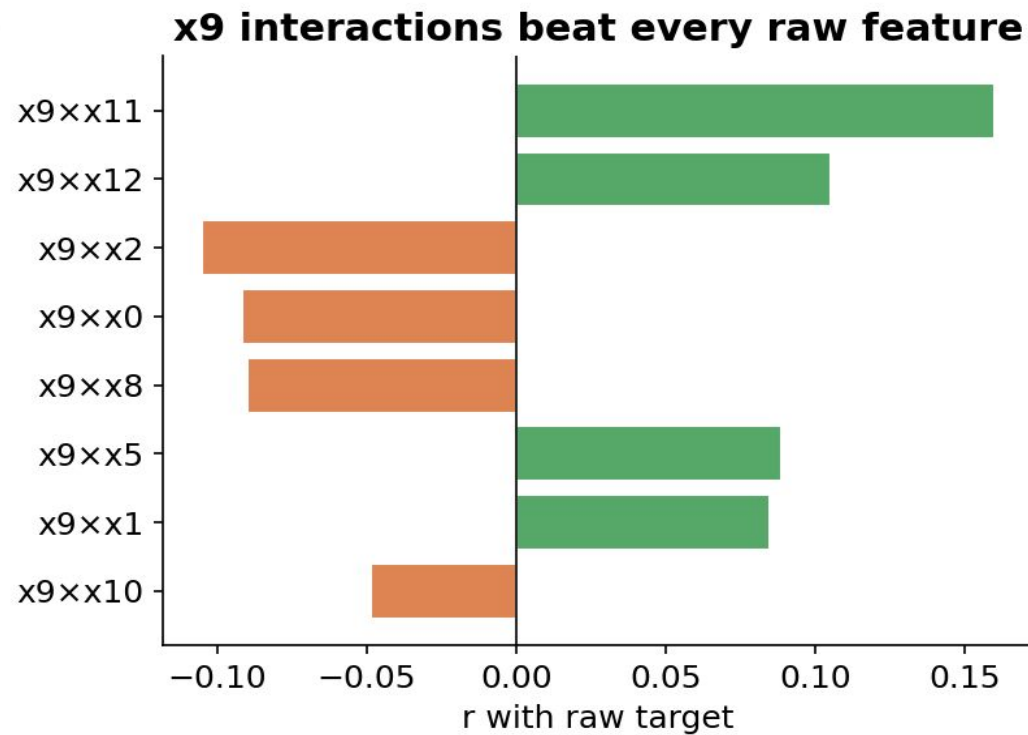
Takeaway: Skew 13.6 and kurtosis 708 vs. 0 for a bell curve. The extreme tail IS the signal the metric rewards — and also what destabilises training.

Raw features barely correlate with the target



Implication: No single feature carries signal (max $|r| \approx 0.06$). Linear models on raw features cap near 0.03 — signal must live in curvature and interactions.

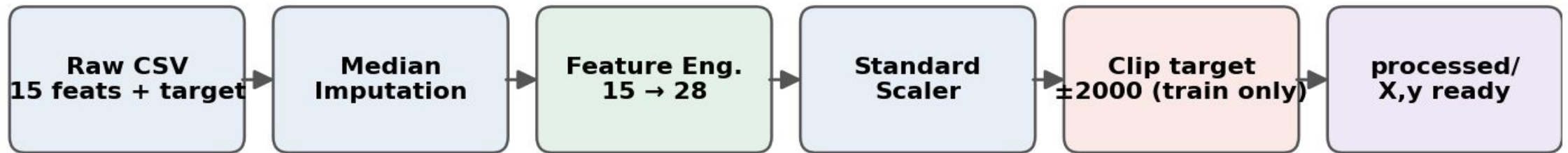
Strongest signals are built using x9



Discovery: x9 × other-features beat every raw feature. x11 is near-zero alone but strongest once multiplied by x9 — a latent 'direction' gate.

Leak-free pipeline, fit on train only

Data preparation pipeline (fit on train only — no leakage)

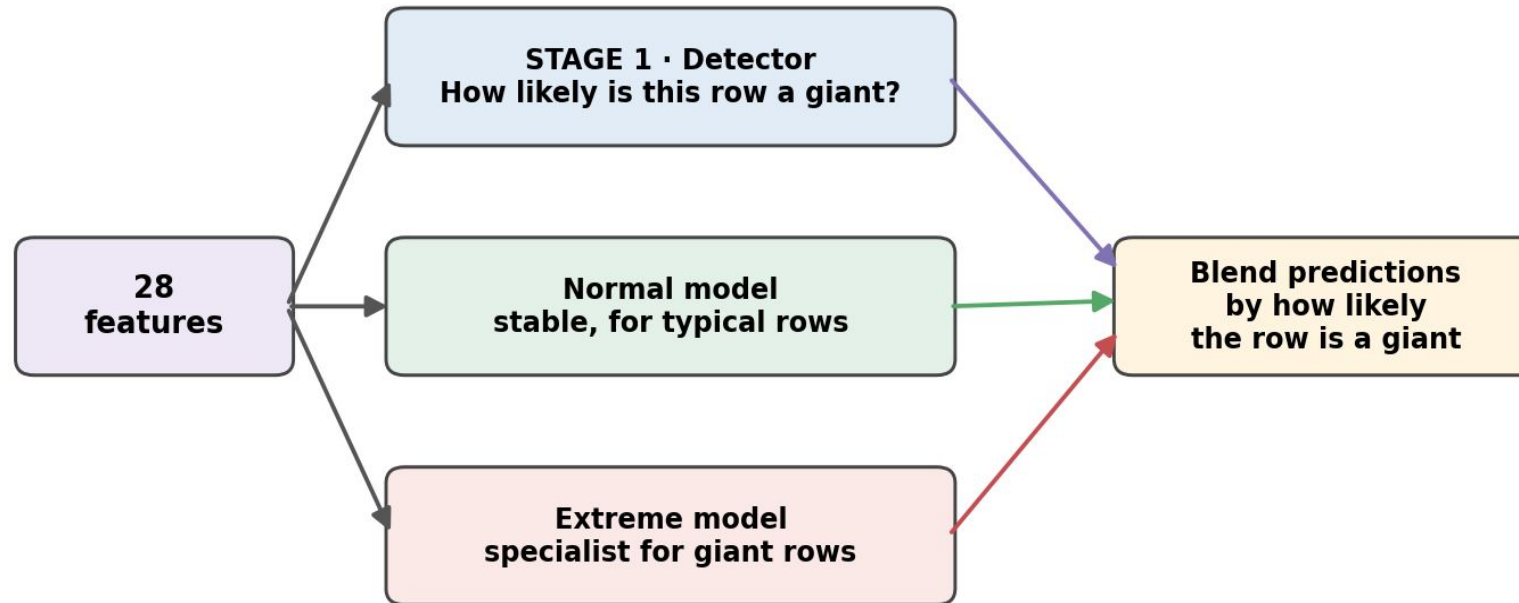


Key rule: clip the target the model LEARNS from to ± 2000 , but always GRADE on raw — tame outliers without changing the yardstick

Core trick: Median impute (features are MCAR & independent), engineer, standard-scale, then CLIP the training target to ± 2000 — tame the giants while the model learns, but always GRADE on raw.

Two-stage: detect first, then route to a specialist

Two-stage model: detect first, then route to the right specialist



Every row gets both predictions; the detector decides how much of each to use — so a likely-normal row stays near the stable prediction, and a likely-giant row reaches toward the tail.

The idea: A single model can't handle both the calm majority and the rare giants. Stage 1 flags likely-giant rows; Stage 2 blends a stable normal model with a specialist extreme model, weighted by how likely each row is to be a giant.

How 5-fold cross-validation actually runs

5-fold cross-validation: refit the whole model on 4 folds, score the held-out fold

round

2,500 rows split into 5 folds (~500 each), shuffled, seed 42

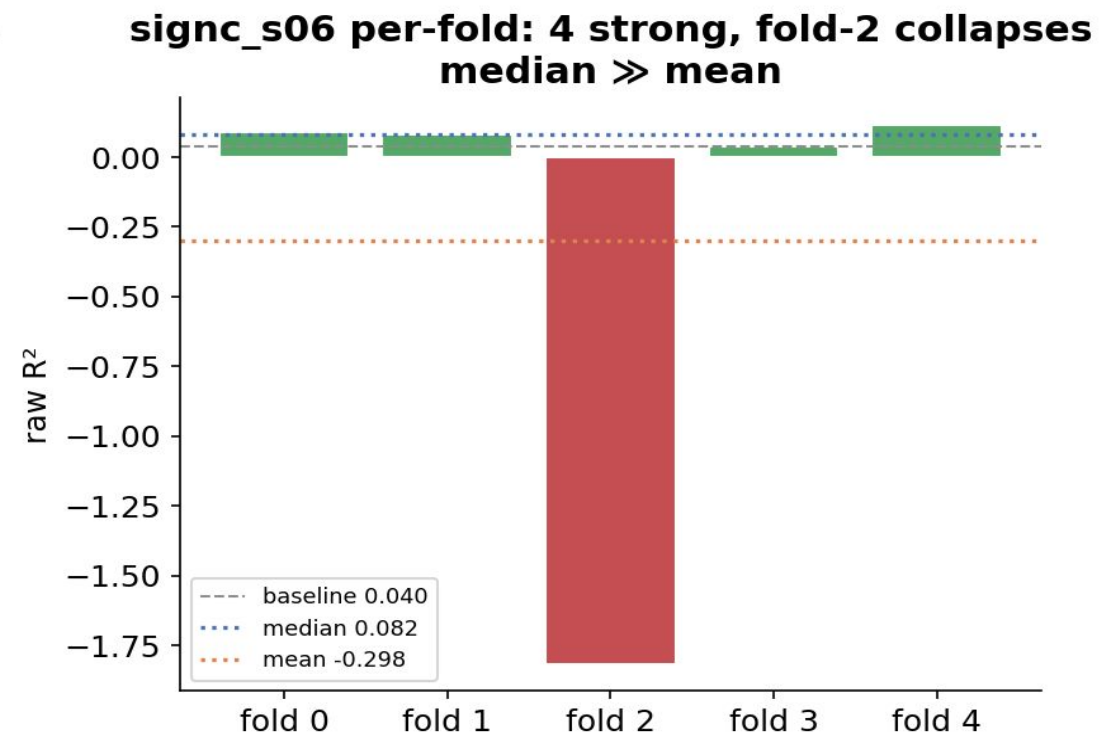
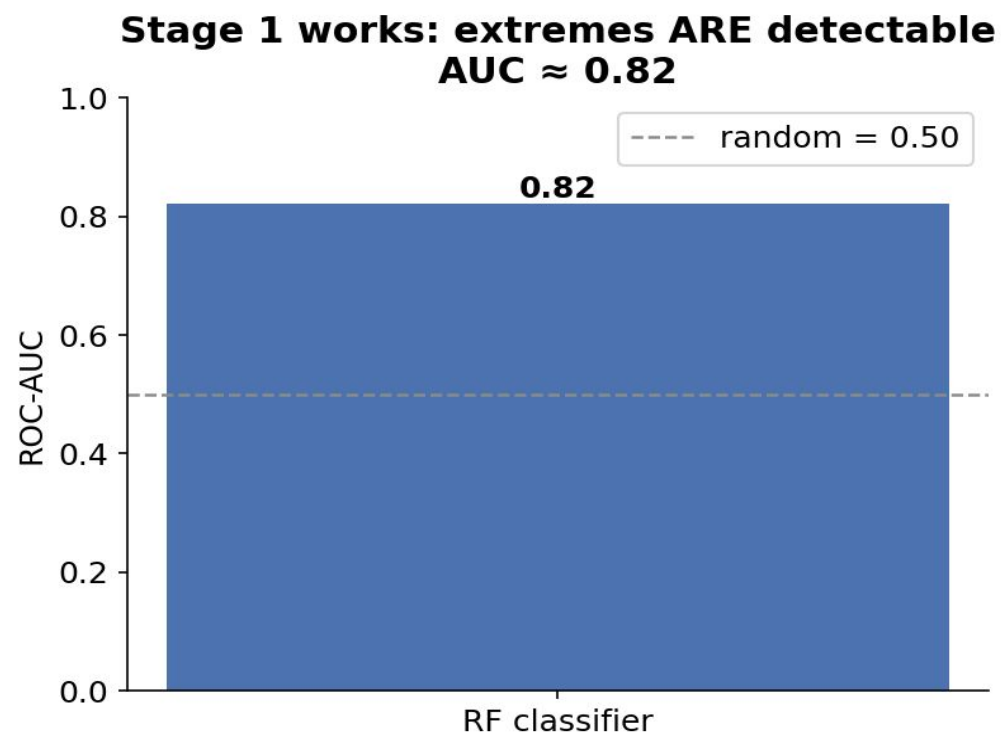
HELD-OUT	train	train	train	train	fold 0: $R^2 = +0.090$
train	HELD-OUT	train	train	train	fold 1: $R^2 = +0.082$
train	train	HELD-OUT	train	train	fold 2: $R^2 = -1.817$
train	train	train	HELD-OUT	train	fold 3: $R^2 = +0.040$
train	train	train	train	HELD-OUT	fold 4: $R^2 = +0.117$

5 scores -> read the MEDIAN, not the mean. median = 0.082 | mean = -0.298 (one collapsed fold) | min = -1.82

Out-of-fold (OOF): stitch the 5 held-out predictions together so every row is predicted by a model that never saw it.

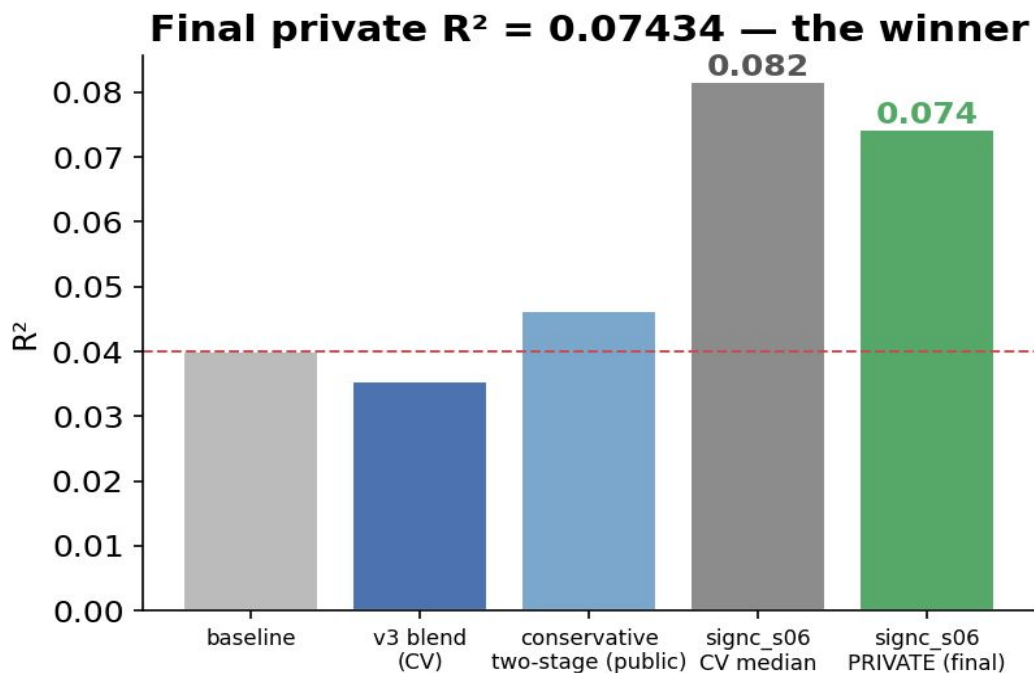
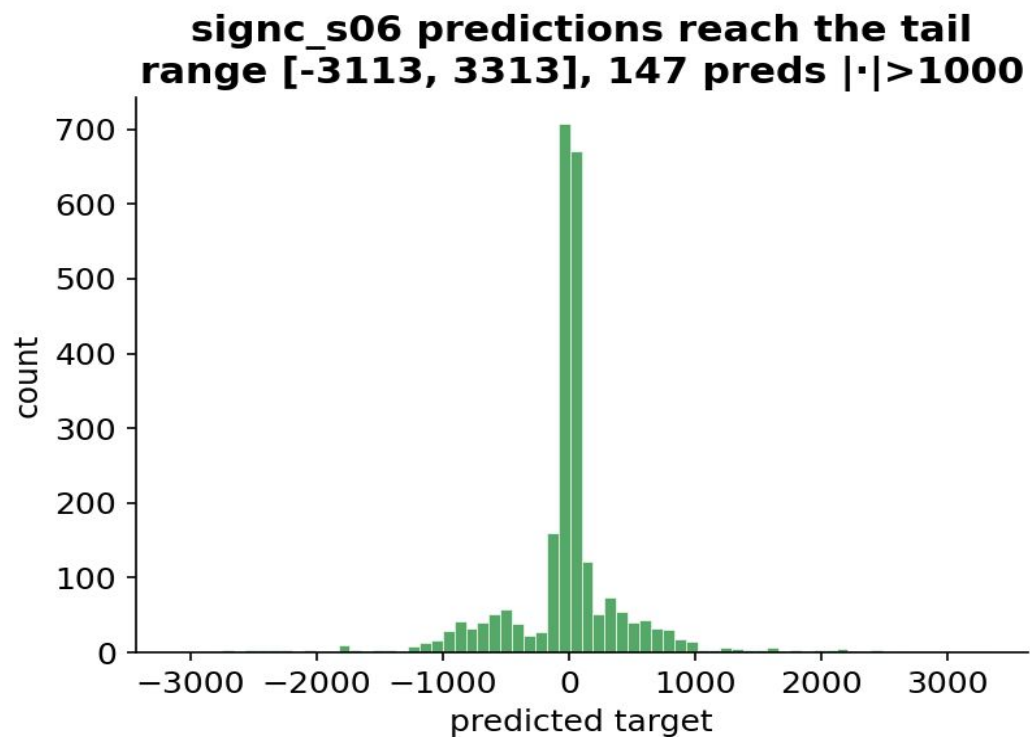
Read the median, not the mean: Each round refits the WHOLE model (backbone + magnitude Ridge + classifier) on 4 folds and scores the 5th. One held-out fold of unseen giants collapses, so the mean misleads — the median is what tracks the real board.

The detector works



Profile of signc_s06: Extremes ARE detectable (AUC 0.82). Four folds reach 0.06–0.12 but fold-2 craters to -1.82 (unseen giants, wrong size). Median 0.082 $\gg$ mean -0.298 — a high-upside, fat-tail bet.

submission_ts_signc_s06.csv → private R^2 0.07434



The payoff: Public scoring punished the bold predictions, but the PRIVATE slice behaved like a typical (good) fold — so the aggressive bet landed at 0.07434, almost exactly its CV median (0.082).